

```

/* Magot Voyage - Service Worker
  Stratégie :
  - Navigations (l'app) : network-first → met à jour index.html quand tu es en l
    et bascule sur la version mise en cache quand tu es hors-ligne.
  - Fichiers locaux (icônes, manifest) : cache-first puis réseau.
  - Requêtes externes (Google Maps/Places/Routes, polices, jsPDF) : laissées au
    elles échouent proprement hors-ligne (l'app gère l'absence de données).

  Pour forcer une mise à jour du cache après un déploiement, change le numéro de
const VERSION = "v524";
const CACHE = "magot-voyage-" + VERSION;
const SHELL = [
  "./",
  "./index.html",
  "./manifest.json",
  "./icon-192.png",
  "./icon-512.png",
  "./icon-maskable-512.png",
  "./apple-touch-icon.png"
];

self.addEventListener("install", (e) => {
  self.skipWaiting();
  e.waitUntil(
    caches.open(CACHE).then((c) => c.addAll(SHELL).catch(() => {}))
  );
});

self.addEventListener("activate", (e) => {
  e.waitUntil(
    caches.keys()
      .then((keys) => Promise.all(keys.filter((k) => k !== CACHE && k !== "mv-osm")
        .then(() => self.clients.claim())
      );
  );
});

self.addEventListener("fetch", (e) => {
  const req = e.request;
  if (req.method !== "GET") return;

  let url;
  try { url = new URL(req.url); } catch (_) { return; }

  // Tuiles/style/polices OSM (OpenFreeMap) + moteur MapLibre (unpkg) : cache-fir
  if (url.hostname === "tiles.openfreemap.org" || url.hostname === "unpkg.com" ||

```

```

e.respondWith(
  caches.open("mv-osm2").then((c) =>
    c.match(req).then((hit) =>
      hit ||
      fetch(req).then((r) => {
        if (r && (r.ok || r.type === "opaque")) { const copy = r.clone(); c.p
        return r;
      }).catch(() => hit) // hors-ligne sans cache → MapLibre gère l'absence
    )
  )
);
return;
}

// Laisse passer tout ce qui est externe (Google, polices, CDN) directement au
if (url.origin !== self.location.origin) return;

// Navigation vers l'app : réseau d'abord (sans cache HTTP), cache en secours (
if (req.mode === "navigate") {
  e.respondWith(
    fetch(req, { cache: "no-store" })
      .then((r) => {
        const copy = r.clone();
        caches.open(CACHE).then((c) => c.put("./index.html", copy)).catch(() =>
        return r;
      })
      .catch(() => caches.match("./index.html").then((r) => r || caches.match("
  );
  return;
}

// Fichiers locaux : cache d'abord, puis réseau (et on met en cache au passage)
e.respondWith(
  caches.match(req).then((cached) =>
    cached ||
    fetch(req)
      .then((r) => {
        if (r && r.status === 200 && r.type === "basic") {
          const copy = r.clone();
          caches.open(CACHE).then((c) => c.put(req, copy)).catch(() => {});
        }
        return r;
      })
      .catch(() => cached)
  )
);
});

```

```
// Permet à l'app de demander une activation immédiate de la nouvelle version.  
self.addEventListener("message", (e) => {  
  if (e.data === "SKIP_WAITING") self.skipWaiting();  
});
```